Lab 4: Frontend & Backend Fundamentals

Lab Overview

This lab provides hands-on experience with building basic frontend and backend applications. You will create simple web interfaces using HTML, CSS, and JavaScript, and develop backend services using Python Flask or Node.js Express. By the end of this lab, you will understand how client-server communication works and be able to build a complete web application.

Topics Covered:

- HTML and CSS basics for frontend
- JavaScript for interactivity
- Backend development with Python Flask or Node.js
- HTTP request/response cycle
- Running applications locally
- Basic API development

Learning Objectives

By the end of this lab, you will be able to:

1. Create basic web pages using HTML and CSS
2. Add interactivity with JavaScript
3. Build a simple backend API using Python Flask or Node.js Express
4. Understand HTTP methods (GET, POST) and status codes
5. Connect frontend and backend applications
6. Run and test web applications locally

Part 1: Frontend Basics - HTML

1.1 Introduction to HTML

HTML (HyperText Markup Language) is the standard markup language for creating web pages. It describes the structure and content of a webpage using elements and tags.

Basic HTML Structure:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Webpage</title>
```

```
</head>
<body>
    <h1>Hello, DevOps!</h1>
    <p>This is my first webpage.</p>
</body>
</html>
```

Create this file as `index.html` and open it in a web browser.

1.2 Common HTML Elements

Headings:

html

```
<h1>Main Heading</h1>
<h2>Subheading</h2>
<h3>Smaller Heading</h3>
<!-- h1 to h6 available -->
```

Paragraphs and Text Formatting:

html

```
<p>This is a paragraph.</p>
<p>This is another paragraph with <strong>bold text</strong> and <em>italic text</em>.</p>
<br> <!-- Line break -->
<hr> <!-- Horizontal rule -->
```

Lists:

html

```
<!-- Unordered list -->
<ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
</ul>

<!-- Ordered list -->
<ol>
    <li>First item</li>
```

```
    <li>Second item</li>
    <li>Third item</li>
</ol>
```

Links:

html

```
<a href="https://www.example.com">Click here</a>
<a href="about.html">About Page</a>
<a href="#section1">Jump to Section 1</a>
```

Images:

html

```
<img src="image.jpg" alt="Description of image">
<img src="https://example.com/logo.png" alt="Logo" width="200">
```

Divs and Spans:

html

```
<div>
    <p>Divs are block-level containers</p>
</div>

<p>This is <span style="color: red;">inline text</span> in a paragraph</p>
```

1.3 HTML Forms

Forms are used to collect user input:

html

```
<form action="/submit" method="POST">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>
    <br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
    <br><br>
```

```
    <label for="message">Message:</label>
    <textarea id="message" name="message" rows="4"></textarea>
    <br><br>

    <button type="submit">Submit</button>
</form>
```

Common Input Types:

- `text`: Single-line text
- `email`: Email address
- `password`: Hidden password
- `number`: Numeric input
- `checkbox`: Checkbox
- `radio`: Radio button
- `date`: Date picker
- `file`: File upload

Part 2: Frontend Styling - CSS

2.1 Introduction to CSS

CSS (Cascading Style Sheets) is used to style and layout HTML elements.

Three Ways to Add CSS:

1. Inline CSS:

html

```
<p style="color: blue; font-size: 16px;">Styled paragraph</p>
```

2. Internal CSS:

html

```
<head>
    <style>
        p {
            color: blue;
            font-size: 16px;
        }
    </style>
</head>
```

3. External CSS (Recommended):

Create `style.css`:

```css
p {
    color: blue;
    font-size: 16px;
}
```

Link in HTML:

```html
<head>
    <link rel="stylesheet" href="style.css">
</head>
```

2.2 CSS Selectors

```css
/* Element selector */
p {
    color: black;
}

/* Class selector */
.highlight {
    background-color: yellow;
}

/* ID selector */
#header {
    font-size: 24px;
}

/* Multiple selectors */
h1, h2, h3 {
    color: navy;
}
```

```
/* Descendant selector */
div p {
    margin-left: 20px;
}
```

2.3 Common CSS Properties

```
CSS
/* Text Styling */
.text-style {
    color: #333;
    font-size: 16px;
    font-family: Arial, sans-serif;
    font-weight: bold;
    text-align: center;
    text-decoration: underline;
}

/* Box Model */
.box {
    width: 300px;
    height: 200px;
    padding: 20px;
    margin: 10px;
    border: 2px solid black;
    background-color: lightblue;
}

/* Display and Layout */
.container {
    display: flex;
    justify-content: center;
    align-items: center;
}

/* Positioning */
```

```
.positioned {
    position: relative;
    top: 10px;
    left: 20px;
}
```

2.4 Creating a Styled Webpage

Create `styled-page.html`:

html

```
<!DOCTYPE html>
<html>
<head>
    <title>DevOps Portfolio</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f0f0f0;
        }

        .container {
            max-width: 800px;
            margin: 50px auto;
            background-color: white;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }

        h1 {
            color: #2c3e50;
            border-bottom: 3px solid #3498db;
            padding-bottom: 10px;
        }
```

```
        .skill {
            display: inline-block;
            background-color: #3498db;
            color: white;
            padding: 5px 15px;
            margin: 5px;
            border-radius: 5px;
        }

        button {
            background-color: #2ecc71;
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background-color: #27ae60;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>DevOps Student Portfolio</h1>
        <h2>About Me</h2>
        <p>I am learning DevOps practices and tools.</p>

        <h2>Skills</h2>
        <span class="skill">Linux</span>
        <span class="skill">Git</span>
        <span class="skill">Shell Scripting</span>
        <span class="skill">Docker</span>

        <h2>Contact</h2>
        <button onclick="alert('Contact feature coming soon!')">Contact Me</button>
```

```
    </div>
</body>
</html>
```

Part 3: Frontend Interactivity - JavaScript

3.1 Introduction to JavaScript

JavaScript adds interactivity to web pages. It runs in the browser and can manipulate HTML and CSS dynamically.

Adding JavaScript to HTML:

1. Inline JavaScript:

html

```html
<button onclick="alert('Hello!')">Click Me</button>
```

2. Internal JavaScript:

html

```html
<script>
    console.log('Page loaded');
    alert('Welcome to my site!');
</script>
```

3. External JavaScript:

Create `script.js`:

javascript

```javascript
console.log('External script loaded');
```

Link in HTML:

html

```html
<script src="script.js"></script>
```

3.2 JavaScript Basics

Variables:

javascript

```javascript
// Variable declarations
let name = "DevOps Student";
const age = 20;
var course = "SE202";  // Older way, use let instead

console.log(name);
console.log(age);
```

Functions:

javascript

```javascript
// Function declaration
function greet(name) {
    return "Hello, " + name + "!";
}

// Function call
let message = greet("Ali");
console.log(message);

// Arrow function (modern way)
const add = (a, b) => a + b;
console.log(add(5, 3));
```

Conditionals:

javascript

```javascript
let score = 85;

if (score >= 90) {
    console.log("Grade: A");
} else if (score >= 80) {
    console.log("Grade: B");
} else {
    console.log("Grade: C");
}
```

Loops:

javascript

```javascript
// For loop
for (let i = 0; i < 5; i++) {
    console.log("Number: " + i);
}

// For...of loop
let skills = ["Linux", "Git", "Docker"];
for (let skill of skills) {
    console.log(skill);
}
```

3.3 DOM Manipulation

The DOM (Document Object Model) allows JavaScript to interact with HTML elements.

Selecting Elements:

javascript

```javascript
// Select by ID
let header = document.getElementById("header");

// Select by class
let buttons = document.getElementsByClassName("btn");

// Select by tag
let paragraphs = document.getElementsByTagName("p");

// Modern selectors
let element = document.querySelector(".class-name");
let elements = document.querySelectorAll(".class-name");
```

Modifying Elements:

javascript

```javascript
// Change text content
document.getElementById("demo").textContent = "New text";

// Change HTML content
```

```javascript
document.getElementById("demo").innerHTML = "<strong>Bold text</strong>";

// Change styles
document.getElementById("demo").style.color = "red";
document.getElementById("demo").style.fontSize = "20px";

// Add/remove classes
element.classList.add("active");
element.classList.remove("hidden");
element.classList.toggle("visible");
```

Event Handling:

javascript

```javascript
// Click event
document.getElementById("myButton").addEventListener("click", function() {
    alert("Button clicked!");
});

// Input event
document.getElementById("nameInput").addEventListener("input", function(event) {
    console.log("Input value:", event.target.value);
});

// Form submit event
document.getElementById("myForm").addEventListener("submit", function(event) {
    event.preventDefault();  // Prevent default form submission
    console.log("Form submitted");
});
```

3.4 Interactive Example

Create `interactive.html`:

html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Interactive Page</title>
```

```
<style>
    body {
        font-family: Arial, sans-serif;
        max-width: 600px;
        margin: 50px auto;
        padding: 20px;
    }

    .task {
        background-color: #f0f0f0;
        padding: 10px;
        margin: 10px 0;
        border-radius: 5px;
    }

    .completed {
        text-decoration: line-through;
        background-color: #d4edda;
    }

    input[type="text"] {
        width: 70%;
        padding: 10px;
        font-size: 16px;
    }

    button {
        padding: 10px 20px;
        font-size: 16px;
        background-color: #007bff;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    button:hover {
```

```
            background-color: #0056b3;
        }
    </style>
</head>
<body>
    <h1>My Task List</h1>

    <input type="text" id="taskInput" placeholder="Enter a new task">
    <button onclick="addTask()">Add Task</button>

    <div id="taskList"></div>

    <script>
        let tasks = [];

        function addTask() {
            let input = document.getElementById("taskInput");
            let taskText = input.value.trim();

            if (taskText === "") {
                alert("Please enter a task!");
                return;
            }

            tasks.push({
                text: taskText,
                completed: false
            });

            input.value = "";
            renderTasks();
        }

        function toggleTask(index) {
            tasks[index].completed = !tasks[index].completed;
            renderTasks();
        }
```

```
        function deleteTask(index) {
            tasks.splice(index, 1);
            renderTasks();
        }

        function renderTasks() {
            let taskList = document.getElementById("taskList");
            taskList.innerHTML = "";

            tasks.forEach((task, index) => {
                let taskDiv = document.createElement("div");
                taskDiv.className = "task";
                if (task.completed) {
                    taskDiv.classList.add("completed");
                }

                taskDiv.innerHTML = `
                    <span onclick="toggleTask(${index})" style="cursor: pointer">
                        ${task.text}
                    </span>
                    <button onclick="deleteTask(${index})"
                            style="float: right; background-color: #dc3545">
                        Delete
                    </button>
                `;

                taskList.appendChild(taskDiv);
            });
        }
    </script>
</body>
</html>
```

Part 4: Backend Development with Python Flask

4.1 Introduction to Flask

Flask is a lightweight Python web framework for building backend applications and APIs.

Installation:

bash

```bash
# Update package list
sudo apt update

# Install Python and pip
sudo apt install python3 python3-pip

# Install Flask
pip3 install flask
```

4.2 Your First Flask Application

Create `app.py`:

python

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello, DevOps!"

@app.route('/about')
def about():
    return "This is the about page"

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Run the application:

bash

```bash
python3 app.py
```

Visit `http://localhost:5000` in your browser.

4.3 Flask Routes and Methods

python

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# GET request
@app.route('/users', methods=['GET'])
def get_users():
    users = [
        {"id": 1, "name": "Ali"},
        {"id": 2, "name": "Sara"}
    ]
    return jsonify(users)

# GET request with parameter
@app.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    user = {"id": user_id, "name": "Ali"}
    return jsonify(user)

# POST request
@app.route('/users', methods=['POST'])
def create_user():
    data = request.get_json()
    new_user = {
        "id": 3,
        "name": data.get('name')
    }
    return jsonify(new_user), 201

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

4.4 Building a Complete API

Create `student_api.py`:

python

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory database (list of dictionaries)
students = [
    {"id": 1, "name": "Ali Ahmed", "course": "SE202", "grade": 85},
    {"id": 2, "name": "Sara Khan", "course": "SE202", "grade": 92}
]

# Get all students
@app.route('/api/students', methods=['GET'])
def get_students():
    return jsonify(students)

# Get single student
@app.route('/api/students/<int:student_id>', methods=['GET'])
def get_student(student_id):
    student = next((s for s in students if s['id'] == student_id), None)
    if student:
        return jsonify(student)
    else:
        return jsonify({"error": "Student not found"}), 404

# Create new student
@app.route('/api/students', methods=['POST'])
def create_student():
    data = request.get_json()

    new_student = {
        "id": len(students) + 1,
        "name": data.get('name'),
        "course": data.get('course'),
        "grade": data.get('grade')
    }
```

```
    students.append(new_student)
    return jsonify(new_student), 201

# Update student
@app.route('/api/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    student = next((s for s in students if s['id'] == student_id), None)

    if not student:
        return jsonify({"error": "Student not found"}), 404

    data = request.get_json()
    student['name'] = data.get('name', student['name'])
    student['grade'] = data.get('grade', student['grade'])

    return jsonify(student)

# Delete student
@app.route('/api/students/<int:student_id>', methods=['DELETE'])
def delete_student(student_id):
    global students
    students = [s for s in students if s['id'] != student_id]
    return jsonify({"message": "Student deleted"}), 200

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

4.5 Testing the API with curl

bash

```
# Get all students
curl http://localhost:5000/api/students

# Get specific student
curl http://localhost:5000/api/students/1

# Create new student
curl -X POST http://localhost:5000/api/students \
```

```
  -H "Content-Type: application/json" \
  -d '{"name":"Ahmed Ali","course":"SE202","grade":88}'

# Update student
curl -X PUT http://localhost:5000/api/students/1 \
  -H "Content-Type: application/json" \
  -d '{"grade":95}'

# Delete student
curl -X DELETE http://localhost:5000/api/students/1
```

Part 5: Alternative Backend - Node.js Express

5.1 Introduction to Node.js and Express

Node.js allows you to run JavaScript on the server side. Express is a popular web framework for Node.js.

Installation:

bash

```
# Install Node.js and npm
sudo apt update
sudo apt install nodejs npm

# Verify installation
node --version
npm --version
```

5.2 Creating an Express Application

bash

```
# Create project directory
mkdir node-backend
cd node-backend

# Initialize npm project
npm init -y
```

```bash
# Install Express
npm install express
```

Create `server.js`:

javascript

```javascript
const express = require('express');
const app = express();

// Middleware to parse JSON
app.use(express.json());

// Routes
app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

app.get('/about', (req, res) => {
  res.json({ message: 'About page', version: '1.0' });
});

// Start server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Run the server:

bash

```bash
node server.js
```

5.3 Building a Complete API with Express

Create `student_api.js`:

javascript

```javascript
const express = require('express');
const app = express();
```

```
app.use(express.json());

// In-memory database
let students = [
    { id: 1, name: "Ali Ahmed", course: "SE202", grade: 85 },
    { id: 2, name: "Sara Khan", course: "SE202", grade: 92 }
];

// Get all students
app.get('/api/students', (req, res) => {
    res.json(students);
});

// Get single student
app.get('/api/students/:id', (req, res) => {
    const student = students.find(s => s.id === parseInt(req.params.id));
    if (!student) {
        return res.status(404).json({ error: 'Student not found' });
    }
    res.json(student);
});

// Create new student
app.post('/api/students', (req, res) => {
    const newStudent = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course,
        grade: req.body.grade
    };
    students.push(newStudent);
    res.status(201).json(newStudent);
});

// Update student
app.put('/api/students/:id', (req, res) => {
    const student = students.find(s => s.id === parseInt(req.params.id));
```

```
  if (!student) {
    return res.status(404).json({ error: 'Student not found' });
  }

  student.name = req.body.name || student.name;
  student.grade = req.body.grade || student.grade;
  res.json(student);
});

// Delete student
app.delete('/api/students/:id', (req, res) => {
  students = students.filter(s => s.id !== parseInt(req.params.id));
  res.json({ message: 'Student deleted' });
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Part 6: Connecting Frontend and Backend

6.1 CORS (Cross-Origin Resource Sharing)

When frontend and backend run on different ports, you need to enable CORS.

For Flask:

bash

```
pip3 install flask-cors
```

python

```
from flask import Flask
from flask_cors import CORS

app = Flask(__name__)
CORS(app)  # Enable CORS for all routes

# Your routes here
```

For Express:

bash

```bash
npm install cors
```

javascript

```javascript
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors());  // Enable CORS
app.use(express.json());

// Your routes here
```

6.2 Complete Frontend-Backend Example

Backend (Flask) - `api_server.py`:

python

```python
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

tasks = []

@app.route('/api/tasks', methods=['GET'])
def get_tasks():
    return jsonify(tasks)

@app.route('/api/tasks', methods=['POST'])
def add_task():
    data = request.get_json()
    task = {
        "id": len(tasks) + 1,
        "text": data.get('text'),
        "completed": False
```

```
    }
    tasks.append(task)
    return jsonify(task), 201

@app.route('/api/tasks/<int:task_id>', methods=['DELETE'])
def delete_task(task_id):
    global tasks
    tasks = [t for t in tasks if t['id'] != task_id]
    return jsonify({"message": "Task deleted"})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Frontend - `frontend.html`:

html

```
<!DOCTYPE html>
<html>
<head>
    <title>Task Manager</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 600px;
            margin: 50px auto;
            padding: 20px;
        }

        .task {
            background-color: #f0f0f0;
            padding: 10px;
            margin: 10px 0;
            border-radius: 5px;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }
```

```
        input[type="text"] {
            width: 70%;
            padding: 10px;
            font-size: 16px;
        }

        button {
            padding: 10px 20px;
            font-size: 16px;
            background-color: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background-color: #0056b3;
        }

        .delete-btn {
            background-color: #dc3545;
            padding: 5px 10px;
        }

        .delete-btn:hover {
            background-color: #c82333;
        }
    </style>
</head>
<body>
    <h1>Task Manager (Connected to Backend)</h1>

    <input type="text" id="taskInput" placeholder="Enter a new task">
    <button onclick="addTask()">Add Task</button>

    <div id="taskList"></div>
```

```
<script>
    const API_URL = 'http://localhost:5000/api/tasks';

    // Load tasks when page loads
    window.onload = function() {
        loadTasks();
    };

    // Load tasks from backend
    function loadTasks() {
        fetch(API_URL)
            .then(response => response.json())
            .then(tasks => {
                displayTasks(tasks);
            })
            .catch(error => {
                console.error('Error loading tasks:', error);
            });
    }

    // Add new task
    function addTask() {
        let input = document.getElementById("taskInput");
        let taskText = input.value.trim();

        if (taskText === "") {
            alert("Please enter a task!");
            return;
        }

        fetch(API_URL, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ text: taskText })
```

```
    })
    .then(response => response.json())
    .then(task => {
        input.value = "";
        loadTasks();  // Reload tasks
    })
    .catch(error => {
        console.error('Error adding task:', error);
    });
}

// Delete task
function deleteTask(taskId) {
    fetch(`${API_URL}/${taskId}`, {
        method: 'DELETE'
    })
    .then(response => response.json())
    .then(result => {
        loadTasks();  // Reload tasks
    })
    .catch(error => {
        console.error('Error deleting task:', error);
    });
}

// Display tasks in UI
function displayTasks(tasks) {
    let taskList = document.getElementById("taskList");
    taskList.innerHTML = "";

    if (tasks.length === 0) {
        taskList.innerHTML = "<p>No tasks yet. Add one above!</p>";
        return;
    }

    tasks.forEach(task => {
        let taskDiv = document.createElement("div");
```

```
            taskDiv.className = "task";

            taskDiv.innerHTML = `
                <span>${task.text}</span>
                <button class="delete-btn" onclick="deleteTask(${task.id})">Delete</button>
            `;

            taskList.appendChild(taskDiv);
        });
    }
  </script>
</body>
</html>
```

Part 7: Practical Exercises

Exercise 1: Create a Personal Portfolio Page

Create an HTML/CSS page with:

1. Your name and photo
2. A list of skills
3. A contact form
4. Styled with CSS
5. Add JavaScript to validate the form

Exercise 2: Build a Calculator API

Create a Flask or Express API with endpoints:

- `/api/add?a=5&b=3` - Addition
- `/api/subtract?a=5&b=3` - Subtraction
- `/api/multiply?a=5&b=3` - Multiplication
- `/api/divide?a=6&b=3` - Division

Exercise 3: Student Management System

Create a full-stack application: **Backend:** API with endpoints for CRUD operations on students
Frontend: HTML page with forms to add, view, update, and delete students

Exercise 4: Weather Dashboard

Create a frontend page that:

1. Has a form to enter city name
2. Displays mock weather data
3. Shows temperature, conditions, humidity
4. Styled attractively with CSS

Lab Assessment Tasks

Task 1: Personal Bio Page

Create `bio.html` with:

- Your information (name, student ID, interests)
- A styled navigation menu
- At least 3 sections with different content
- Responsive CSS styling
- A working contact form (frontend only)

Task 2: REST API for Books

Create a backend API (`books_api.py` or `books_api.js`) with:

- GET `/api/books` - List all books
- GET `/api/books/:id` - Get specific book
- POST `/api/books` - Add new book
- PUT `/api/books/:id` - Update book
- DELETE `/api/books/:id` - Delete book

Each book should have: id, title, author, year

Task 3: Interactive Quiz Page

Create `quiz.html` with:

- At least 5 multiple-choice questions about DevOps
- JavaScript to check answers
- Display score at the end
- Styled with CSS

Task 4: Todo List with Backend

Create a complete todo application:

- Backend API with Flask/Express

- Frontend HTML page
- Ability to add and delete tasks
- Tasks stored in backend
- Frontend communicates with backend via fetch API

Task 5: Documentation

Create `api-documentation.md` that:

- Explains the purpose of your API
- Lists all endpoints with examples
- Shows example requests and responses
- Explains how to run your application

Submit all files with a README explaining how to run each component.